🪟 Zaawansowany Spark SQL

Window functions, Catalyst Optimizer i sztuka niekopania sobie dołka wydajnościowego ⚡

Python w Zastosowaniach Big Data | Wykład 6

Prowadzący: Pavlo Melnyk

📋 Plan wykładu

1. Window functions — supermoc, o której za mało mówiliśmy
2. UDF-y — i dlaczego prawie zawsze warto ich unikać
3. Catalyst Optimizer — jak Spark "myśli" o Twoim zapytaniu
4. Tungsten — silnik, który robi całą brudną robotę
5. Partycjonowanie: `repartition` vs `coalesce`
6. Czytanie `.explain()` jak profesjonalista
7. Cache, performance tips i najczęstsze pułapki

> 🎯 **Po tym wykładzie:** Napiszesz zapytanie z window functions, przeczytasz jego plan wykonania i wytłumaczysz, dlaczego jest szybkie (albo wolne) 🔍

💡 Dlaczego to ważne?

Wyobraź sobie system wypożyczalni rowerów miejskich 🚲 — jak Veturilo w Warszawie czy Nextbike w Poznaniu.

Menedżer pyta Cię:

> "Która stacja jest najbardziej obciążona? Jak zmienia się liczba wypożyczeń dzień po dniu? Czy da się wykryć, że rower stoi w jednym miejscu podejrzanie długo?"

Zwykły `groupBy` da Ci **jedną liczbę** na stację. Ale menedżer chce **rankingu**, **sum narastających** i porównań **"poprzedni vs następny przejazd"**.

> 🎯 To dokładnie robota dla **window functions** — i tym właśnie zajmiemy się dziś, na prawdziwym (syntetycznym) zbiorze danych o rowerach miejskich!

🔄 Szybkie przypomnienie z Wykładu 5

Z poprzedniego wykładu (Spark SQL i DataFrames) pamiętamy:

- **DataFrame API** i **Spark SQL** to dwa interfejsy do tego samego silnika
- `select`, `filter`, `withColumn`, `groupBy`, `agg`, `join` — codzienny chleb
- Transformacje są **leniwe** (lazy), akcje (`show`, `count`, `collect`) je uruchamiają

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName("Wyklad6").getOrCreate()
```

> Dziś budujemy na tym fundamencie — czas na **window functions** i zajrzenie "pod maskę" Sparka 🔧

🚲 Zbiór danych na dziś

Dane o wypożyczeniach rowerów miejskich (~200 000 przejazdów, generowane syntetycznie):

Kolumna	Opis
`trip_id`	Unikalny identyfikator przejazdu
`start_station`	Stacja startowa
`end_station`	Stacja końcowa
`start_time`	Czas rozpoczęcia przejazdu
`end_time`	Czas zakończenia przejazdu
`duration_min`	Czas trwania przejazdu (minuty)
`user_type`	`"abonament"` lub `"jednorazowy"`
`distance_km`	Dystans przejazdu (km)

```
import random
random.seed(42)  # 👈 powtarzalność wyników
```

🪟 Window Functions — czym różnią się od `groupBy`?

`groupBy` **redukuje** liczbę wierszy. Window function **nie redukuje nic** — dodaje kolumnę obliczoną na podstawie "okna" powiązanych wierszy.

```
# groupBy — jedna liczba na stację
trips.groupBy("start_station").count()

# window function — każdy przejazd DOSTAJE swój ranking, ale nic nie znika
from pyspark.sql.window import Window

window_spec = Window.partitionBy("start_station").orderBy(F.desc("duration_min"))
trips.withColumn("ranking", F.rank().over(window_spec))
```

> 🎯 **Mantra:** `Window = partitionBy (podział na grupy) + orderBy (kolejność w grupie)`

Window functions — ranking stacji (przykład prowadzony)

Zadanie: Chcemy ranking stacji startowych wg liczby wypożyczeń.

```
from pyspark.sql.window import Window
from pyspark.sql import functions as F

# Krok 1: liczba przejazdów per stacja
station_counts = trips.groupBy("start_station").agg(
    F.count("trip_id").alias("num_trips")
)

# Krok 2: okno bez partycji (ranking globalny) + sortowanie malejąco
global_window = Window.orderBy(F.desc("num_trips"))

ranked_stations = station_counts.withColumn(
    "rank", F.rank().over(global_window)
)

ranked_stations.orderBy("rank").show(10)
```

> 💡 Brak `partitionBy` = jedno wielkie okno na całym DataFrame!

🪟 `row_number` vs `rank` vs `dense_rank`

```
window_spec = Window.partitionBy("start_station").orderBy(F.desc("duration_min"))

result = trips.select(
    "start_station", "trip_id", "duration_min",
    F.row_number().over(window_spec).alias("row_num"),
    F.rank().over(window_spec).alias("rank"),
    F.dense_rank().over(window_spec).alias("dense_rank"),
)
```

Funkcja	Remisy	Luki w numeracji
`row_number`	Zawsze unikalny numer	—
`rank`	Ten sam numer dla remisu	✅ Tak (1, 2, 2, 4)
`dense_rank`	Ten sam numer dla remisu	❌ Nie (1, 2, 2, 3)

> 🎯 Do "top-3 stacje" najczęściej używamy `row_number` — zawsze dostaniesz dokładnie 3 wiersze!

🤔 Think About It

Pytanie: Chcemy **dokładnie 3** najbardziej ruchliwe stacje w każdym mieście. Której funkcji użyjesz?

```
window_spec = Window.partitionBy("city").orderBy(F.desc("num_trips"))

# A) F.rank()
# B) F.dense_rank()
# C) F.row_number()
```

`lag` i `lead` — poprzedni i następny przejazd

Chcemy policzyć **przerwę czasową** między kolejnymi przejazdami z tej samej stacji:

```
from pyspark.sql.functions import lag, lead

station_time_window = Window.partitionBy("start_station").orderBy("start_time")

with_gaps = trips.select(
    "start_station", "trip_id", "start_time",
    F.lag("start_time", 1).over(station_time_window).alias("poprzedni_start"),
    F.lead("start_time", 1).over(station_time_window).alias("nastepny_start"),
)

with_gaps = with_gaps.withColumn(
    "przerwa_min",
    (F.col("start_time").cast("long") - F.col("poprzedni_start").cast("long")) / 60
)
```

> 💡 `lag` = "co było przed" 🚗↩️, `lead` = "co będzie po" 🚗↪️. Duża `przerwa_min` może oznaczać, że rower stał bezczynnie!

🪟 Running totals — `rowsBetween`

Suma narastająca (running total) liczby przejazdów per stacja per dzień:

```
daily_window = (
    Window.partitionBy("start_station")
    .orderBy("trip_date")
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)  # 👈 od początku do "teraz"
)

with_running_total = daily_trips.withColumn(
    "cumulative_trips",
    F.sum("daily_trips").over(daily_window)
)
```

`rowsBetween` definiuje **zakres okna** względem bieżącego wiersza:

Zakres	Znaczenie
`unboundedPreceding, currentRow`	Od początku partycji do teraz (running total)
`-2, currentRow`	2 poprzednie wiersze + bieżący (średnia krocząca)
`unboundedPreceding, unboundedFollowing`	Cała partycja (jak `groupBy`, ale bez redukcji)

🛠️ UDF — kiedy wbudowane funkcje nie wystarczą

```python
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

@udf(returnType=StringType())
def kategoria_dystansu(km: float) -> str:
    if km is None:
        return "brak_danych"
    if km < 2:
        return "krotki"
    if km < 8:
        return "sredni"
    return "dlugi"

trips.withColumn("kategoria", kategoria_dystansu(F.col("distance_km")))
```

> ⚠️ Zanim napiszesz UDF — sprawdź, czy `pyspark.sql.functions` już tego nie ma! (`F.when()` załatwiłby powyższy przykład bez UDF)

⚠️ UDF — dlaczego Python UDF jest wolny?

```
                Szybka ścieżka – wbudowane funkcje F.*
DataFrame ──▶ [Catalyst] ──▶ [Tungsten / JVM] ──▶ Wynik      ⚡ Szybko!

                Wolna ścieżka – Python UDF
DataFrame ──▶ [JVM] ──serializacja──▶ [Python] ──▶ [JVM] ──▶ Wynik
                        per KAŻDY wiersz ↑ to boli 😭
```

Podejście	Wydajność	Uwagi
Wbudowane `F.*`	⚡⚡⚡	Catalyst zna ich wnętrze — może optymalizować
Pandas UDF (vectorized)	⚡⚡	Operuje na całych partiach danych (Arrow)
Klasyczny Python UDF	⚡	Catalyst traktuje je jak "czarną skrzynkę" — brak optymalizacji!

> 🎯 **Zasada #1:** Python UDF = Spark nie wie, co robisz w środku. Zero predicate pushdown, zero column pruning dla tej kolumny!

🧠 Catalyst Optimizer — jak Spark "myśli"

Każde zapytanie — DataFrame API czy SQL — przechodzi przez **te same 4 fazy**:

```
1. Parsed Logical Plan     – dosłownie to, co napisałeś
        ↓
2. Analyzed Logical Plan   – sprawdzone typy, kolumny, tabele
        ↓
3. Optimized Logical Plan  – predicate pushdown, column pruning, constant folding...
        ↓
4. Physical Plan(s)        – konkretne strategie wykonania (+ wybór najtańszej)
```

> 💡 Dlatego `df.filter(...)` i `spark.sql("SELECT ... WHERE ...")` dają **identyczny** plan wykonania — oba przechodzą przez ten sam Catalyst!

🧠 Predicate pushdown w praktyce

Predicate pushdown = filtry są przesuwane **jak najbliżej źródła danych**, żeby czytać mniej.

```
# Piszemy:
trips.join(stations, "start_station").filter(F.col("user_type") == "abonament")

# Catalyst optymalizuje logicznie do:
trips.filter(F.col("user_type") == "abonament").join(stations, "start_station")
# 👈 Filtr PRZED joinem — mniej wierszy do złączenia!
```

Inne kluczowe optymalizacje Catalyst:

- **Column pruning** — czyta tylko kolumny faktycznie użyte w zapytaniu
- **Constant folding** — oblicza stałe wyrażenia raz, na etapie planowania
- **Join reordering** — dobiera kolejność joinów minimalizującą koszt

🔍 `.explain()` — zaglądamy pod maskę

```
result = (
    trips
    .filter(F.col("user_type") == "abonament")
    .groupBy("start_station")
    .agg(F.count("trip_id").alias("num_trips"))
    .orderBy(F.desc("num_trips"))
)

result.explain(True)  # 👈 True = pokaż wszystkie 4 fazy planu
```

```
== Physical Plan ==
*(2) Sort [num_trips DESC]
+- Exchange rangepartitioning(num_trips DESC)        ← shuffle!
   +- *(1) HashAggregate [start_station], [count(trip_id)]
      +- *(1) Filter (user_type = abonament)
         +- FileScan parquet [start_station,trip_id,user_type]  ← column pruning!
```

> 🎯 Widzisz `Exchange` w planie? To **shuffle** — najdroższa operacja w Sparku. Warto wiedzieć, kiedy się pojawia!

🔍 Czytanie planu — na co patrzeć

Element planu	Co oznacza
`FileScan`	Odczyt danych ze źródła — sprawdź, czy widać tylko potrzebne kolumny
`Filter`	Warunek `WHERE` — im wcześniej w planie, tym lepiej
`Exchange`	**Shuffle** — dane wędrują między executorami (kosztowne!)
`HashAggregate`	Fizyczna implementacja `groupBy` + `agg`
`BroadcastHashJoin`	Join z małą tabelą wysłaną do każdego executora (tanio!)
`SortMergeJoin`	Join wymagający shuffle i sortowania obu stron (drożej)
`Window`	Fizyczna implementacja window function — zwykle wymaga sortowania w partycji

> 💡 Im mniej `Exchange` w planie, tym szybciej poleci Twoje zapytanie!

⚙️ Tungsten — silnik wykonawczy pod Catalystem

Tungsten to projekt Sparka odpowiedzialny za **fizyczne** wykonanie planu — bliżej sprzętu niż Catalyst.

Co robi Tungsten?

- **Binary in-memory format** — dane trzymane jako bajty, bez narzutu obiektów Javy/Pythona
- **Whole-stage code generation** — generuje bajtkod JVM dla całego fragmentu planu naraz (mniej wywołań wirtualnych!)
- **Zarządzanie pamięcią off-heap** — mniej pracy dla Garbage Collectora

```
Catalyst  → "CO obliczyć i w jakiej kolejności" (logika, plan)
Tungsten  → "JAK to fizycznie policzyć szybko" (bajty, kod maszynowy, pamięć)
```

> 🎯 Dlatego DataFrame/Spark SQL jest zwykle **szybszy niż RDD** — RDD nie korzysta z Tungstena tak samo efektywnie!

📦 Partycjonowanie — dlaczego to ważne

Spark dzieli DataFrame na **partycje** — niezależne kawałki przetwarzane równolegle przez executory.

```
print(trips.rdd.getNumPartitions())  # np. 200 (domyślna liczba partycji shuffle)
```

- **Za mało partycji** → nie wykorzystujesz wszystkich rdzeni/executorów 🐌
- **Za dużo partycji** → narzut na zarządzanie małymi zadaniami (task scheduling overhead) 🐌
- **W sam raz** → każdy executor ma robotę mniej więcej równej wielkości ⚡

> 💡 Dobra reguła startowa: 2-4× liczba rdzeni w klastrze jako liczba partycji.

📦 `repartition()` vs `coalesce()`

```
# repartition — ZAWSZE robi shuffle, może zwiększyć LUB zmniejszyć liczbę partycji
trips_more = trips.repartition(50)

# repartition po kolumnie — dane z tym samym kluczem trafiają do tej samej partycji
trips_by_station = trips.repartition("start_station")  # 👈 świetne przed joinem/window!

# coalesce — zmniejsza liczbę partycji BEZ pełnego shuffla (łączy sąsiednie partycje)
trips_fewer = trips.coalesce(10)
```

Operacja	Shuffle?	Kiedy używać
`repartition(n)`	✅ Zawsze	Chcesz **zwiększyć** partycje lub równomiernie je rozłożyć
`repartition("kolumna")`	✅ Zawsze	Przed joinem/window na tej kolumnie — dane trafią razem
`coalesce(n)`	⚠️ Zwykle nie	Chcesz **zmniejszyć** partycje (np. przed zapisem do pliku)

> ⚠️ `coalesce()` do **zwiększania** partycji praktycznie nie działa — użyj wtedy `repartition()`!

🤔 Think About It

Pytanie: Masz DataFrame z 500 partycjami, ale chcesz zapisać wynik jako **1 plik CSV**. Której operacji użyjesz?

A) `trips.repartition(1).write.csv(...)`
B) `trips.coalesce(1).write.csv(...)`
C) Obie zadziałają identycznie szybko

💾 Cache i persist — kiedy warto

```
# Cache w pamięci (domyślnie MEMORY_AND_DISK)
popular_trips = trips.filter(F.col("user_type") == "abonament")
popular_trips.cache()      # 👈 leniwe – realny cache przy pierwszej akcji
popular_trips.count()      # 👈 teraz dane faktycznie są w cache'u

# ... wielokrotne użycie popular_trips ...

popular_trips.unpersist()  # 👈 zwolnij pamięć, gdy skończysz!
```

Poziom `persist()`	Opis
`MEMORY_ONLY`	Tylko RAM — najszybciej, ryzyko utraty przy braku pamięci
`MEMORY_AND_DISK`	RAM + dysk jako zapasowa opcja — domyślne dla `.cache()`
`DISK_ONLY`	Tylko dysk — gdy naprawdę mało RAM-u

> 🎯 Cachuj tylko wtedy, gdy dany DataFrame jest **używany wielokrotnie** w dalszych krokach!

⚡ Performance tip — unikaj niepotrzebnych shuffli

Operacje generujące shuffle: `groupBy` + agregacja, `join` (bez broadcast), `orderBy`, `repartition`, `distinct`, window functions.

```
# ❌ Filtr PO grupowaniu — zbędna praca na danych, które i tak odpadną
trips.groupBy("start_station").count().filter(F.col("count") > 100)

# ✅ Filtruj wcześnie — mniej danych do shuffle
trips.filter(F.col("distance_km") > 0) \
     .groupBy("start_station").count() \
     .filter(F.col("count") > 100)
```

> 💡 **Złota zasada Sparka:** filtruj wcześnie, shuffluj późno!

💻 Live Demo — kompletny przykład (rowery miejskie)

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window

spark = SparkSession.builder.appName("Wyklad6Demo").getOrCreate()

data = [
    (1, "Centrum", "Stare Miasto", 12, "abonament", 2.3, "2026-11-01"),
    (2, "Stare Miasto", "Centrum", 8, "jednorazowy", 1.1, "2026-11-01"),
    (3, "Centrum", "Politechnika", 20, "abonament", 4.7, "2026-11-02"),
    (4, "Politechnika", "Centrum", 15, "abonament", 4.5, "2026-11-02"),
    (5, "Centrum", "Stare Miasto", 10, "jednorazowy", 2.1, "2026-11-02"),
]

cols = ["trip_id", "start_station", "end_station", "duration_min",
        "user_type", "distance_km", "trip_date"]
trips = spark.createDataFrame(data, cols)
```

💻 Demo — ranking i running total

```
daily = trips.groupBy("start_station", "trip_date").agg(
    F.count("trip_id").alias("daily_trips")
)

running_window = (
    Window.partitionBy("start_station")
    .orderBy("trip_date")
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)
)

result = daily.withColumn(
    "cumulative_trips", F.sum("daily_trips").over(running_window)
).withColumn(
    "rank_dnia", F.rank().over(Window.partitionBy("trip_date").orderBy(F.desc("daily_trips")))
)

result.orderBy("start_station", "trip_date").show()
```

💻 Demo — sprawdźmy plan wykonania

```
result.explain()
```

```
== Physical Plan ==
*(4) Project [start_station, trip_date, daily_trips, cumulative_trips, rank_dnia]
+- Window [rank(daily_trips) windowspecdefinition(trip_date, daily_trips DESC)]
   +- *(3) Sort [trip_date ASC]
      +- Exchange hashpartitioning(trip_date)          ← shuffle nr 2
         +- Window [sum(daily_trips) ... unboundedpreceding, currentrow]
            +- *(2) Sort [start_station ASC, trip_date ASC]
               +- Exchange hashpartitioning(start_station)  ← shuffle nr 1
                  +- HashAggregate [start_station, trip_date], [count(trip_id)]
```

> 🎯 Każde `Window` z **inną** kombinacją `partitionBy` wymaga **osobnego** shuffla!

⚠️ Częste pułapki

#	Pułapka	Dlaczego boli	Rozwiązanie
1	Wiele okien z różnym `partitionBy` w jednym zapytaniu	Każde to osobny shuffle	Grupuj window functions z tym samym oknem razem
2	Python UDF zamiast `F.*`	Serializacja JVM↔Python per wiersz	Sprawdź `pyspark.sql.functions` najpierw
3	`repartition()` zamiast `coalesce()` przy zmniejszaniu	Zbędny pełny shuffle	`coalesce()` do zmniejszania partycji
4	Brak `orderBy` w oknie dla `lag` / `lead`	Losowa, niedeterministyczna kolejność	Zawsze definiuj `orderBy` w oknie czasowym
5	Cache bez `unpersist()`	Wyciek pamięci na executorach	Zwalniaj cache, gdy niepotrzebny
6	Ignorowanie `.explain()` przed wdrożeniem	Niespodzianki wydajnościowe na produkcji	Zawsze sprawdź plan na reprezentatywnej próbce

✅ Best practices — ściągawka

1. 🪟 **Window = partitionBy + orderBy** — zawsze pytaj "w jakich grupach?" i "w jakiej kolejności?"
2. 🐍 **Unikaj Python UDF** — najpierw sprawdź wbudowane funkcje `F.*`
3. 🔍 `.explain()` przed uruchomieniem na dużych danych — szukaj zbędnych `Exchange`
4. 📦 `coalesce()` **do zmniejszania,** `repartition()` **do zwiększania/rozkładania**
5. 💾 **Cache** tylko to, co używasz wielokrotnie — i pamiętaj o `unpersist()`
6. ⚡ **Filtruj wcześnie, shuffluj późno**

```
# ✅ Wzorzec produkcyjny
result = (
    spark.read.parquet("trips/")
    .filter(F.col("trip_date") >= "2026-11-01")   # filtr wcześnie
    .repartition("start_station")                  # rozłóż pod window/join
    .withColumn("rank", F.rank().over(window_spec))
)
result.explain()  # sprawdź plan!
```

📝 Podsumowanie — kluczowe wnioski

1. ✅ **Window functions** dodają kolumnę obliczoną na "oknie" wierszy, **bez** redukcji liczby wierszy
2. ✅ `rank`, `dense_rank`, `row_number`, `lag`, `lead` i `rowsBetween` rozwiązują 90% analiz rankingowych i czasowych
3. ✅ **Catalyst Optimizer** planuje zapytanie w 4 fazach; **Tungsten** wykonuje je fizycznie i szybko
4. ✅ `.explain()` to najlepszy przyjaciel przy diagnozowaniu wydajności — szukaj `Exchange`!
5. ✅ `repartition()` vs `coalesce()` — inny koszt, inne zastosowanie

🔮 Co dalej?

Wykład 7 (17.11.2026): Przygotowanie danych do ML — Feature Engineering 🧪

Nauczymy się, jak przekształcić surowe dane (w tym te same przejazdy rowerowe!) w cechy gotowe do zasilenia modelu Machine Learning: kodowanie zmiennych kategorycznych, skalowanie, obsługa braków danych.

Ćwiczenia dziś: Window functions na pełnym zbiorze ~200 000 przejazdów rowerowych — rankingi stacji, running totals, wykrywanie anomalii "rebalancingu" rowerów 🚲🔍

❓ Pytania?

Dziękuję za uwagę! 🎉

Zapraszam na ćwiczenia — czas ubrudzić sobie ręce kodem 💻